

Software Systems

Session 5

Modeling using UML

Y. Raghu Reddy

Software Engineering Research Center
IIIT Hyderabad, India

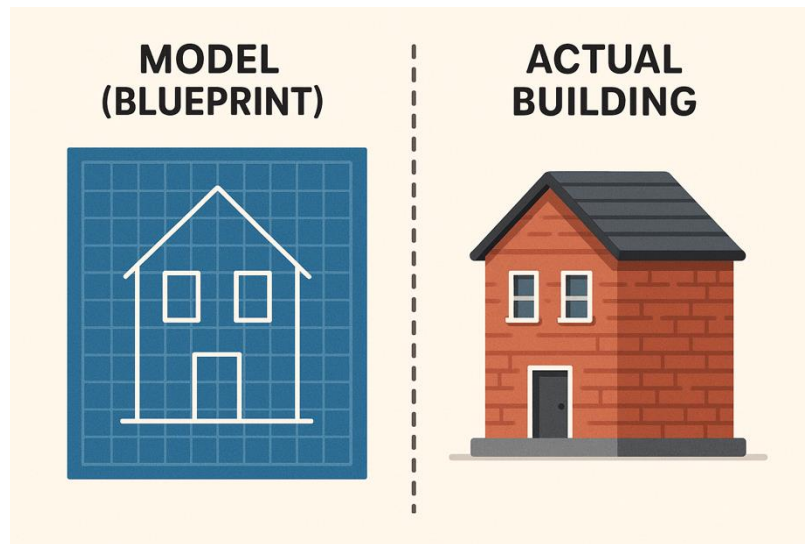


From Requirements to Design Models

- SDLC → establishes lifecycle and processes
- Project Planning → defines “when” and “who”
- Requirements Engineering → defines “what” the system must do
- Requirements Modeling → structures and validates “what”
- **Now: UML Modeling → defines “how” the system will be structured and behave**

Models

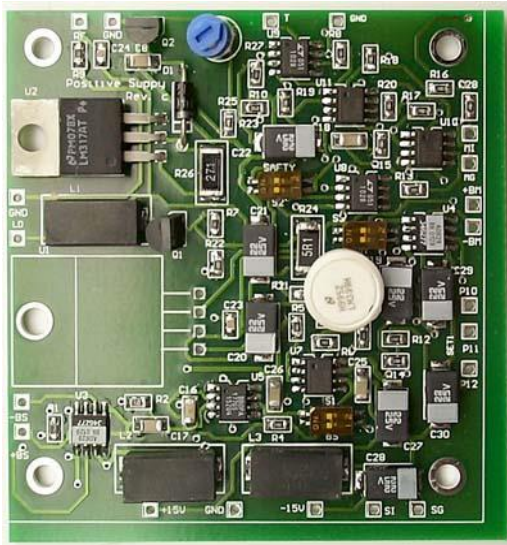
- A *model* is a description of something
 - “a pattern for something to be made” (Merriam-Webster)



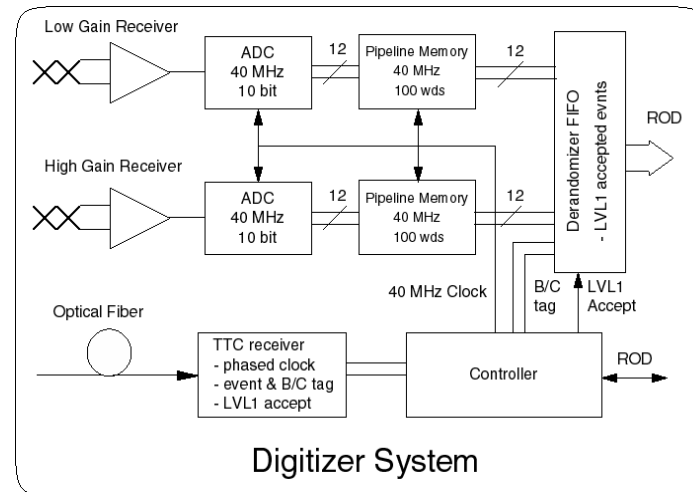
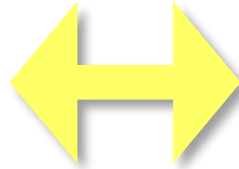
- model $\neq$ thing that is modeled
 - ▣ The Map is Not The Territory

Engineering Models

A reduced representation of some system that highlights the properties of interest from a given viewpoint



Modeled system



Functional Model

- ◆ We don't see everything at once
- ◆ We use a representation (notation) that is easily understood for the purpose on hand

Models

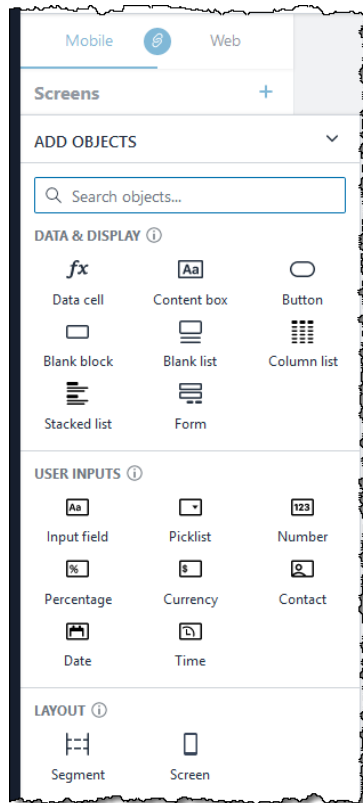
- How do we model?
- Modeling Maturity Level
 - Level 0: No specification
 - Level 1: Textual
 - Level 2: Text with Diagrams
 - Level 3: Models with Text
 - Level 4: Precise Models

Modeling Languages

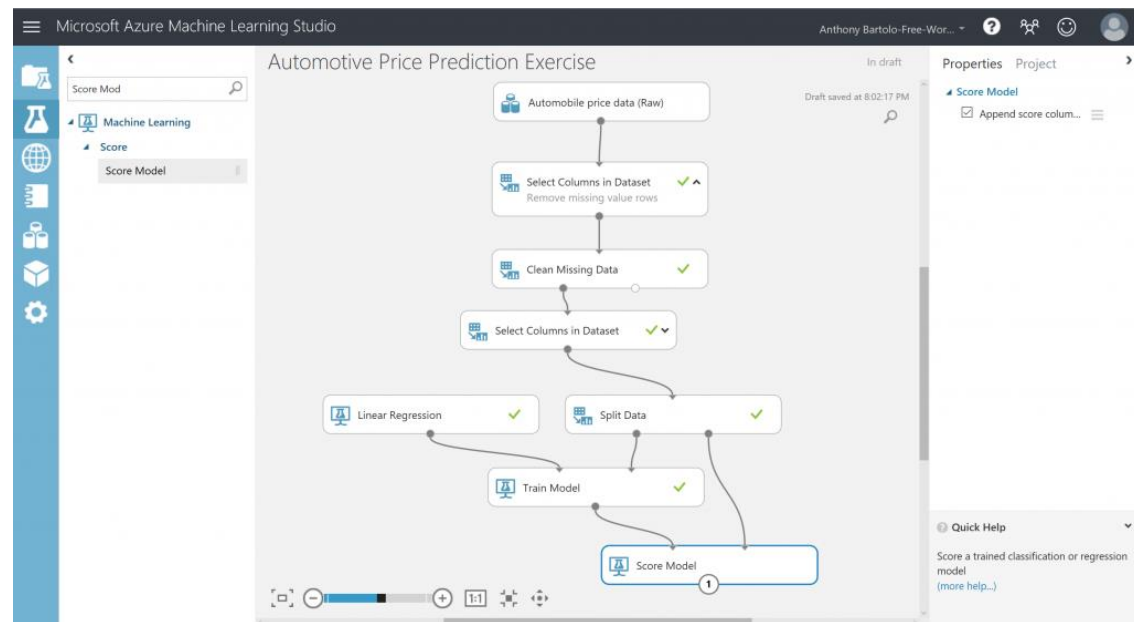
Largely classified into two types

- Domain-Specific Languages (DSLs)
 - Languages designed to model a certain domain
 - Examples: HTML, SQL, etc.
- General Purpose Modeling Languages (GPLs)
 - Languages can be applied to any domain for modeling
 - Examples: **UML**, XML, etc.

Some Examples

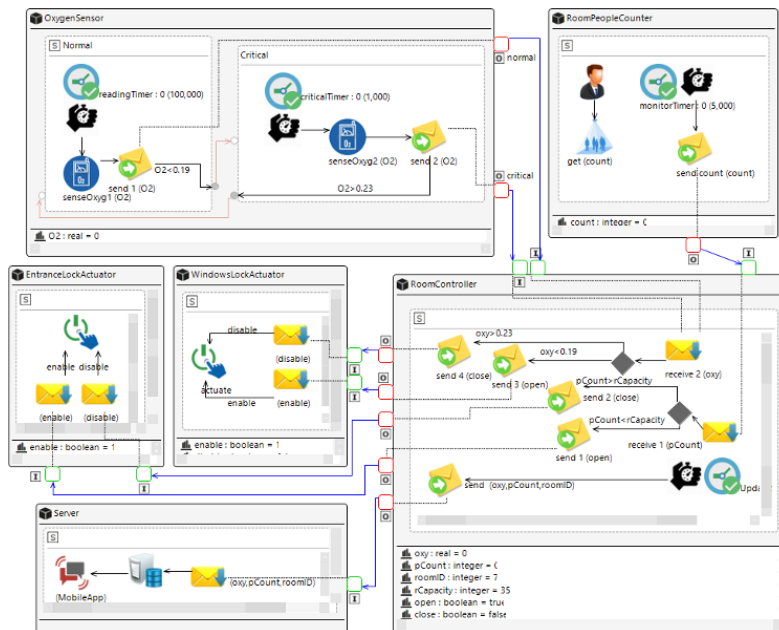


AWS Honeycode



Azure Machine Learning Studio

Some Examples - We can create our own too



CAPS modeling for IoT

```
configuration Robot
{
    instance robot : RobotControl
    instance sdist : DistanceSensor
    instance scoll : CollisionSensor
    instance motion : MotionControl
    instance left_wheel : WheelControl
    instance right_wheel : WheelControl

    connector robot.rangefinder => sdist.data
    connector robot.bumper => scoll.data
    connector robot.platform => motion.ctrl
    connector motion.left => left_wheel.ctrl
    connector motion.right => right_wheel.ctrl
}
```

ThingML modeling

In the Context of Our Course
(GPL – UML)

Unified Modeling Language (UML): Brief History



- No common language to model until 1996
- UML developed by industry consortium in 1997
 - Introduction of OOP in IT dates back to 1960's
 - Required a standard representation: **OMG**
 - Three Amigos: Grady Booch, Ivar Jacobson and James Rumbaugh
- Based on multiple prior visual modeling languages
- Goal was to have a single language that could cover large number of SE tasks
- Current version of UML: 2.5.1 (as of Dec 2017)

Unified Modeling Language (UML)

- Notation for OO Modeling
 - Use object orientation as basis
 - Model a system as collection of objects that interact with each other
- Graphical diagrams as a way to model systems
 - More clear (imprecise) than natural language (too detailed)
 - Capture an overall view of the system
 - Independent of language or technology

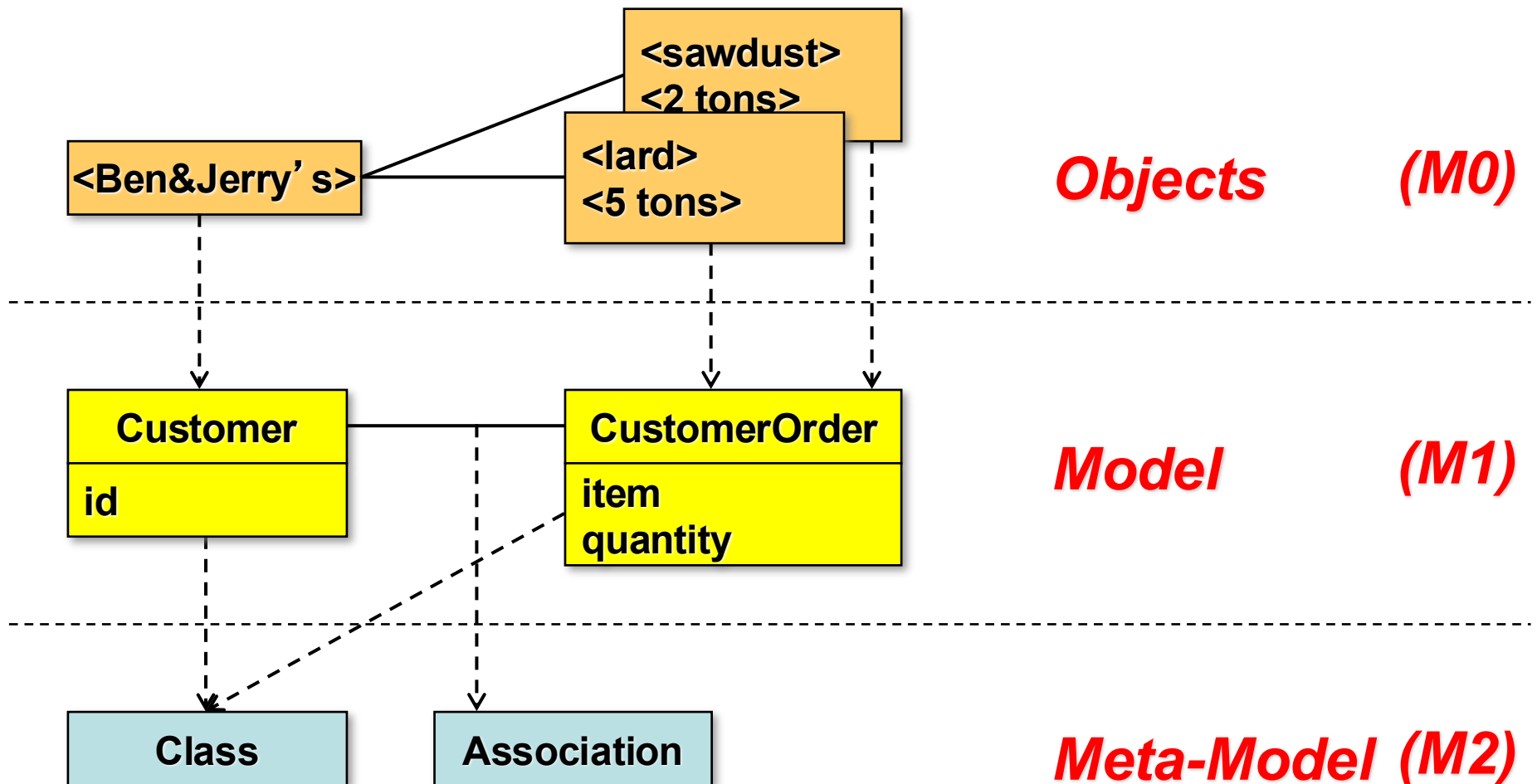
What UML is not?

- Not an OO Method or Process
- Not a visual programming language
- Not a tool specification

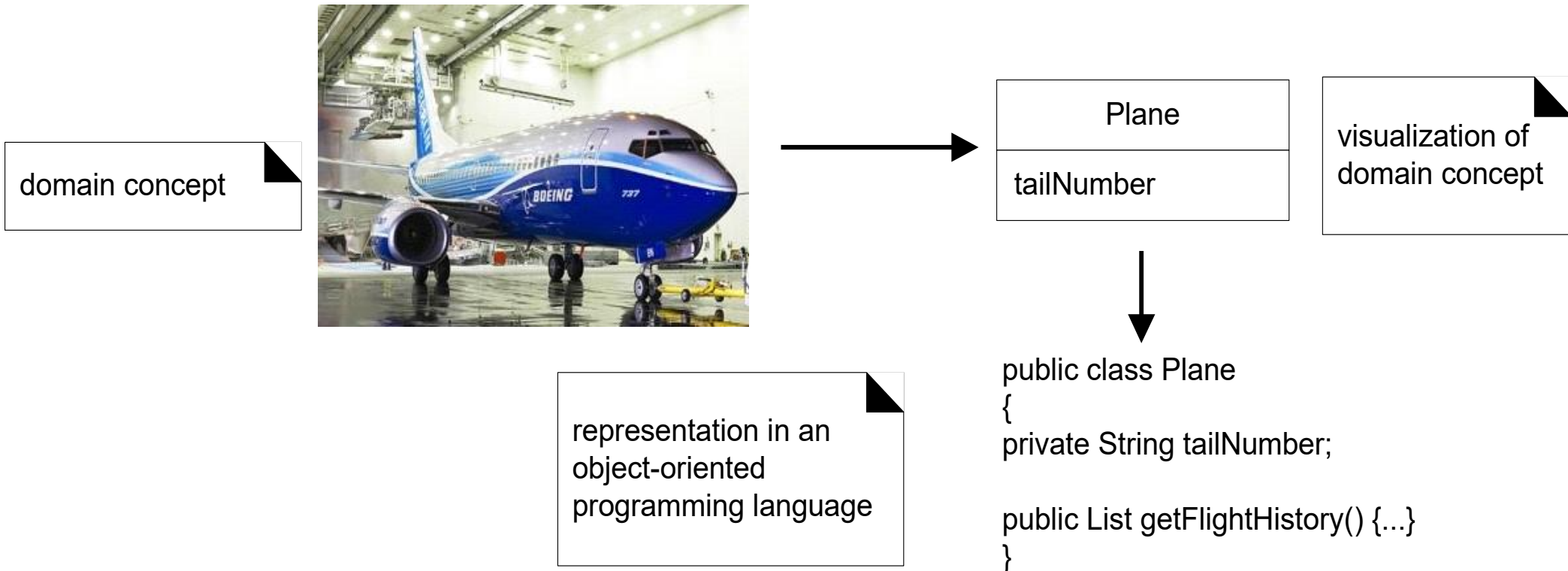


Models and Meta-Models

Meta-models are simply Models of Models

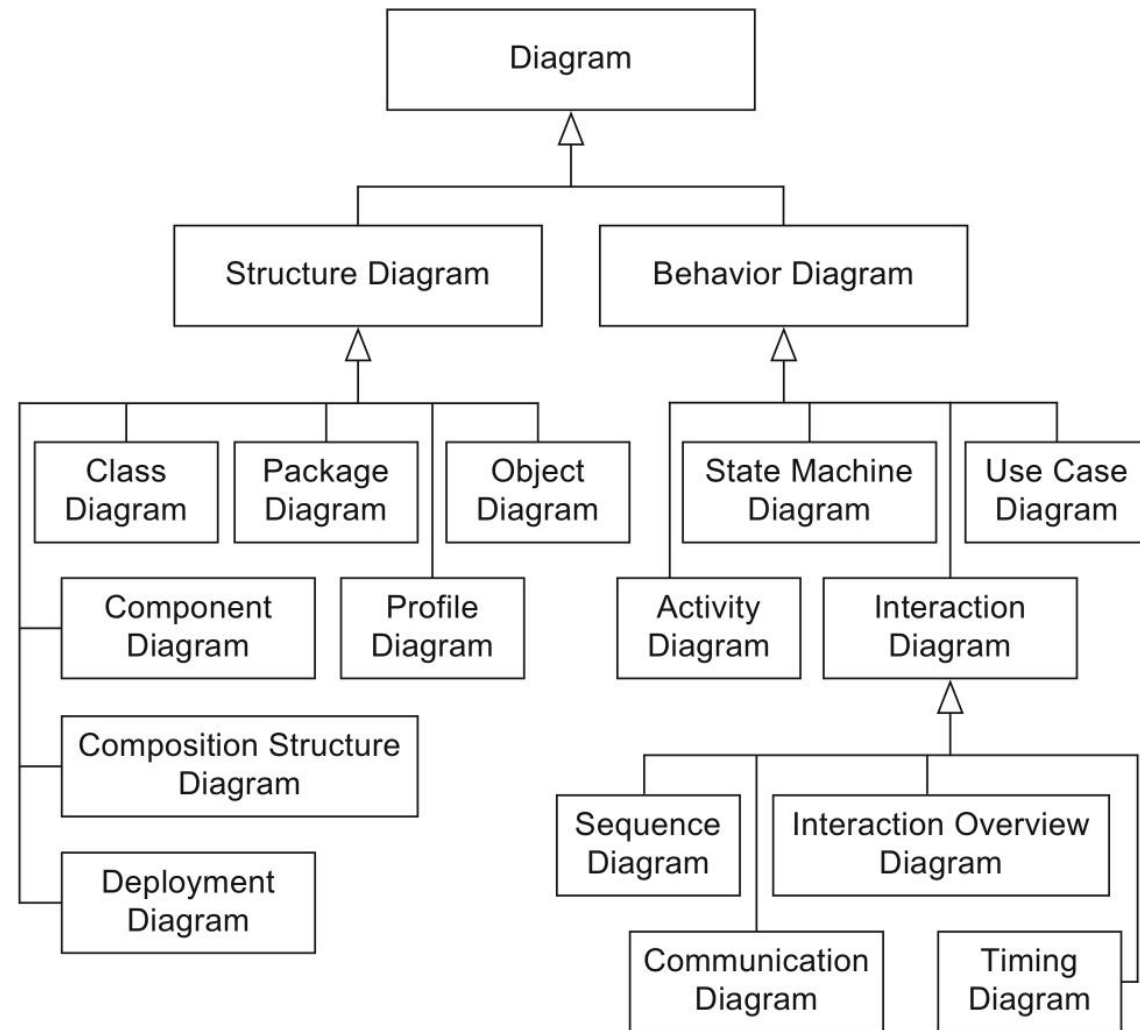


Domain concept
vs.
design representation of domain concept
vs.
code representation of domain concept

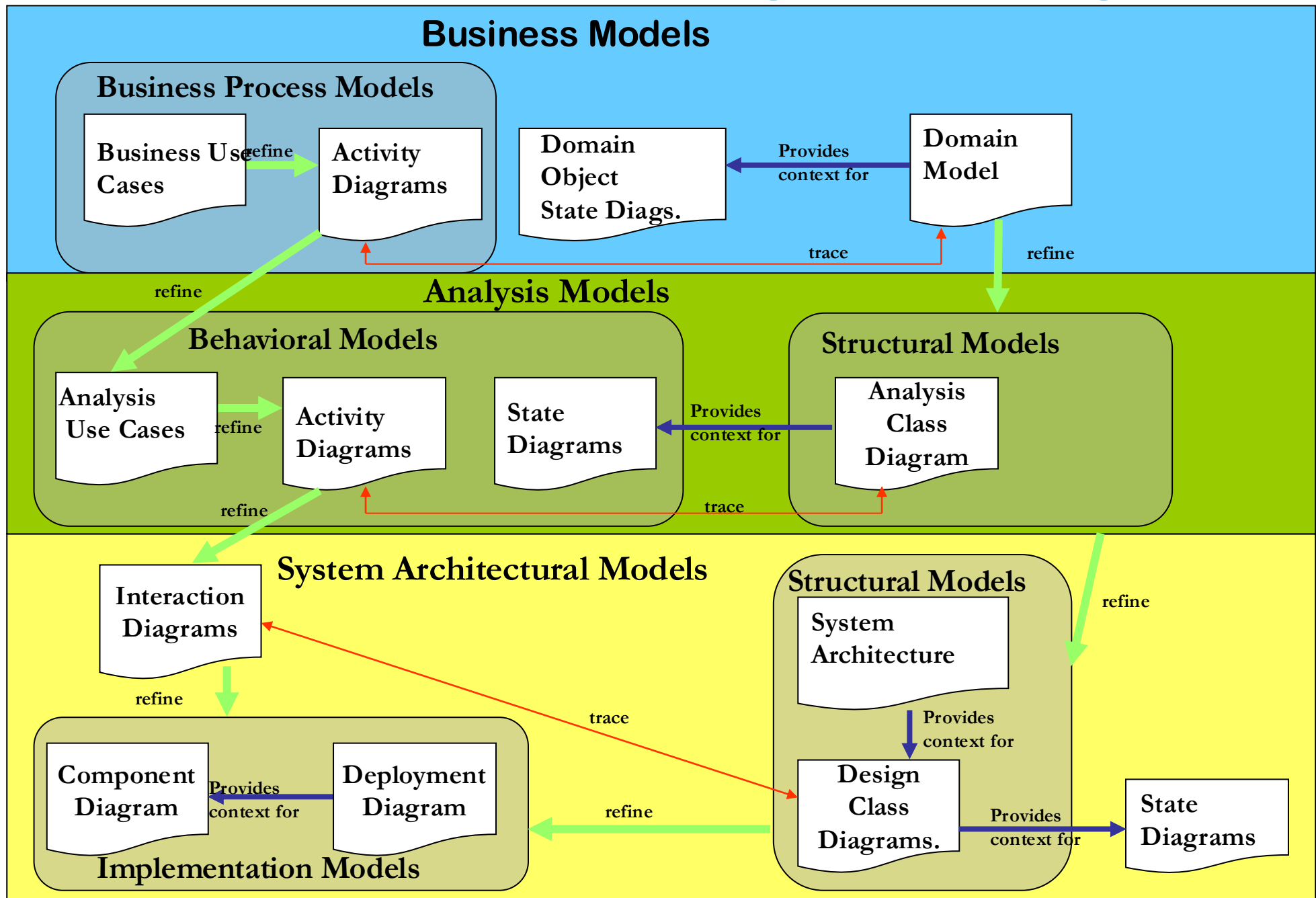


UML Diagrams

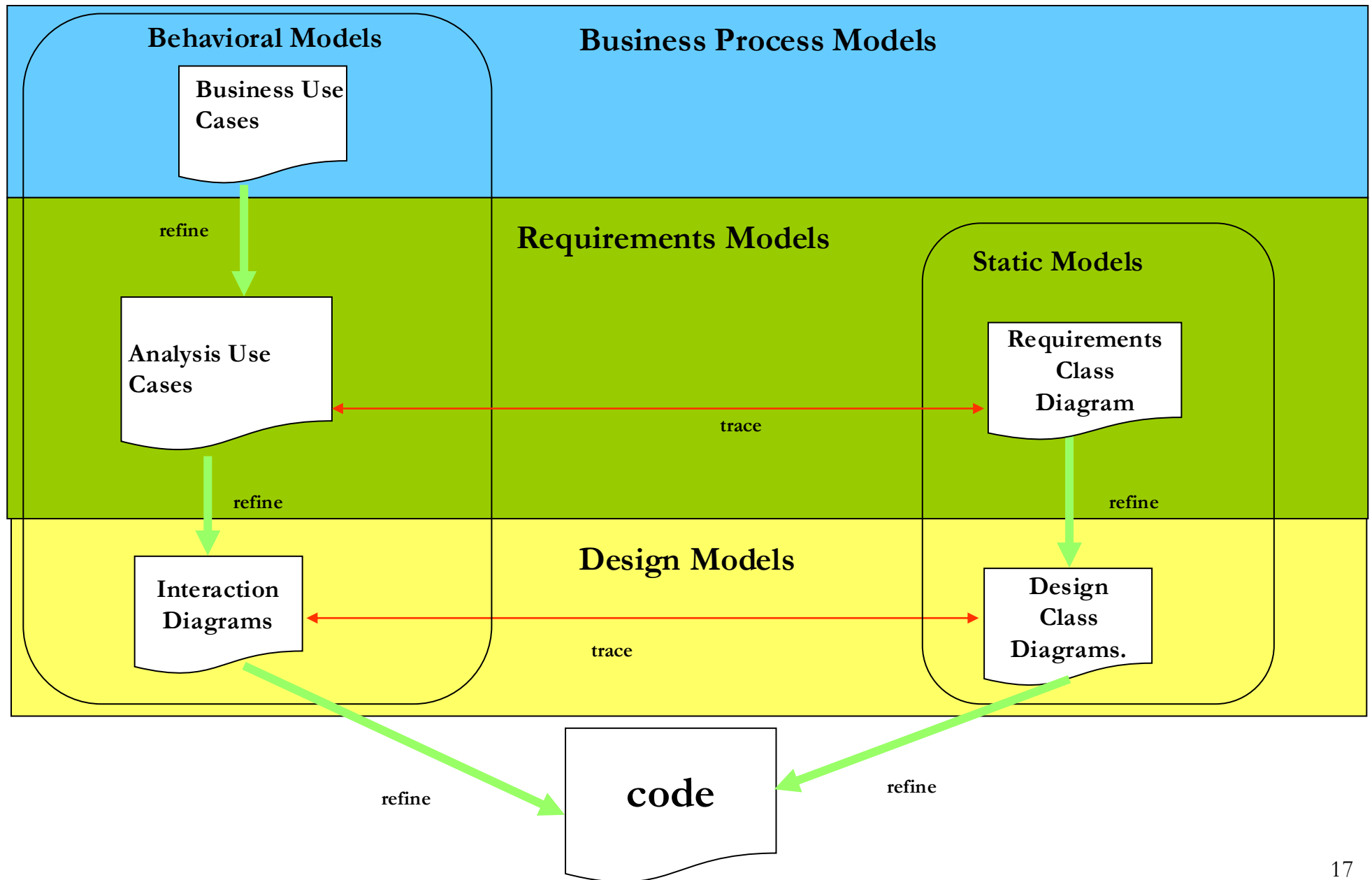
- 14 different diagrams
- Structure diagrams for capturing static aspects of system
- Behavior diagrams for capturing dynamic aspect of system



A “Full” Process (using UML diagrams)



An “UltraLite” Process



Static Vs Dynamic Models

Static Model

- Describes the static structure of a system
- One of the most common diagrams: *class diagrams*

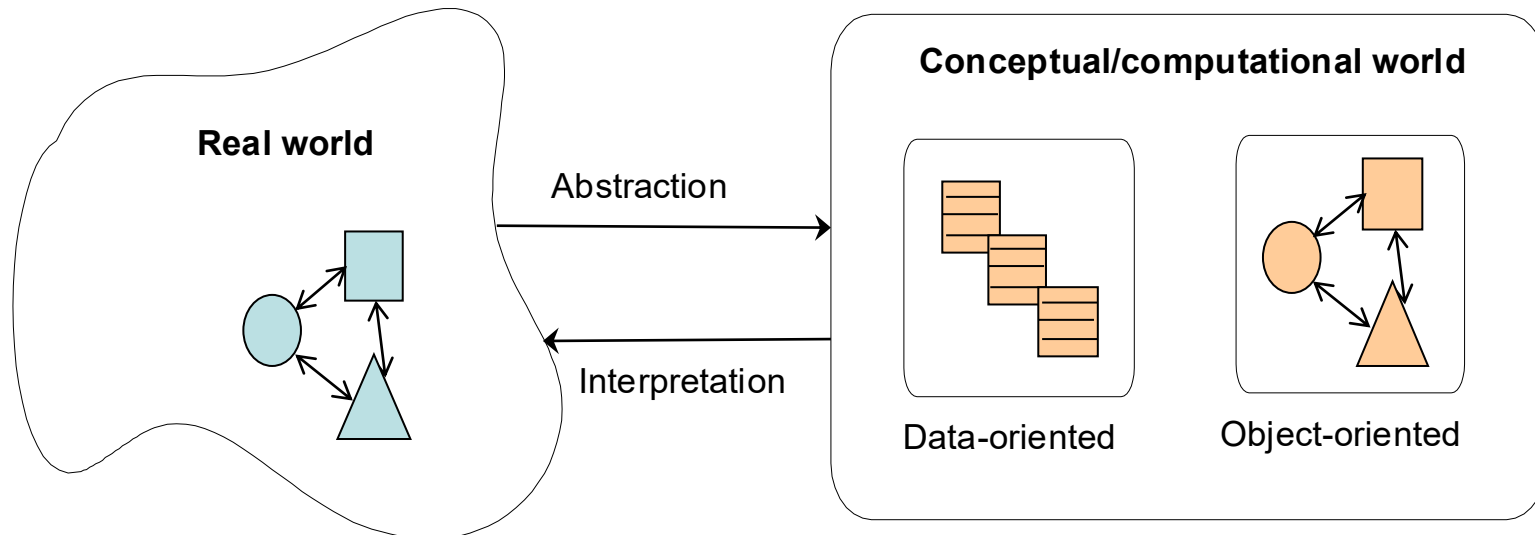
Dynamic Model

- Captures the dynamic behavior of a system
- Developed with help of state chart diagrams, sequence diagrams, etc.



Object-Oriented Modeling

- Uses object-orientation as a basis of modeling
- Models a system as a set of objects that interact with each others
- No semantic gap (or impedance mismatch)
- Seamless development process



Key Ideas of O-O Modeling

- Abstraction
- Encapsulation
- Relationship
 - Association: relationship between objects
 - Inheritance: mechanism to represent similarity among objects
- Object-oriented
 - = object (class) + inheritance + message send

Objects vs. Classes

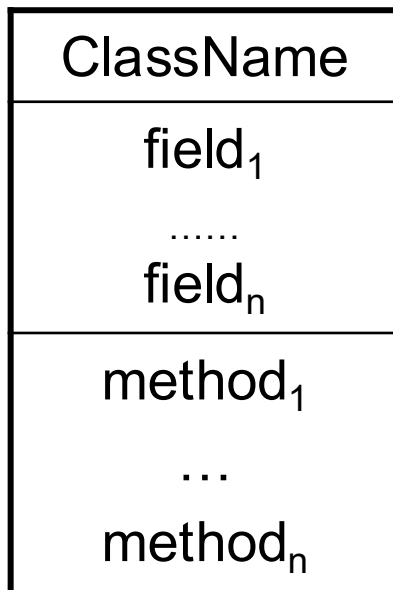
	Interpretation in the Real World	Representation in the Model
Object	An <i>object</i> represents anything in the real world that can be distinctly identified.	An <i>object</i> has an identity, a state, and a behavior.
Class	A <i>class</i> represents a set of objects with similar characteristics and behavior. These objects are called <i>instances</i> of the class.	A <i>class</i> characterizes the structure of states and behaviors that are shared by all of its instances.

UML Class Diagram

- Most common diagram in OO modeling
- Describes the static structure of a system
- Consist of:
 - Nodes representing classes
 - Links representing of relationships among classes
 - Inheritance
 - Association, including aggregation and composition
 - Dependency

Notation for Classes

- The UML notation for classes is a rectangular box with as many as three compartments.



The top compartment show the class name.

The middle compartment contains the declarations of the fields, or *attributes*, of the class.

The bottom compartment contains the declarations of the methods of the class.

Example

Point

Point
x
y
move

Point
- x: int - y: int
+ move(dx: int, dy: int): void

Field and Method Declarations in UML

- Field declarations
 - birthday: Date
 - +duration: int = 100
 - -students[1..MAX_SIZE]: Student
- Method declarations
 - +move(dx: int, dy: int): void
 - +getSize(): int

Visibility	Notation
public	+
protected	#
package	~
private	-

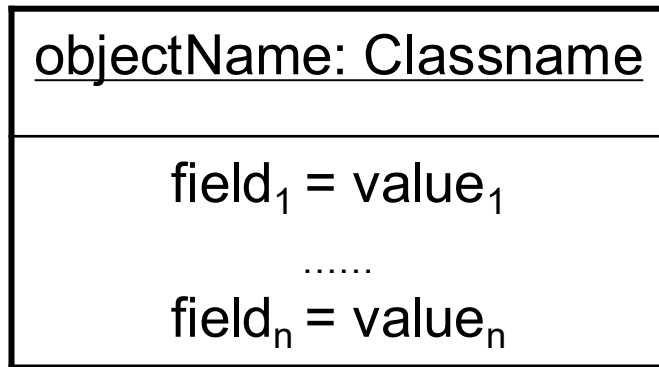
Practice activity

- Model a UML class diagram for the following Java code.

```
class Person {  
    private String name;  
    private Date birthday;  
    public String getName() {  
        // ...  
    }  
    public Date getBirthday() {  
        // ...  
    }  
}
```

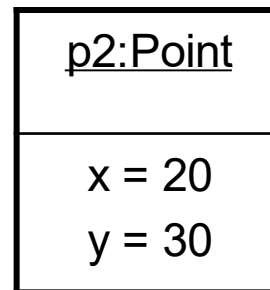
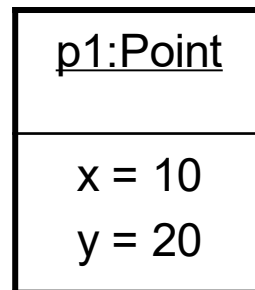
Notation for Objects

Rectangular box with one or two compartments



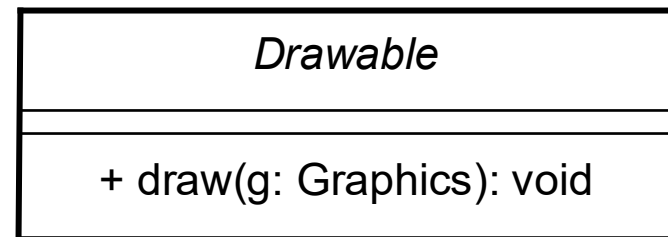
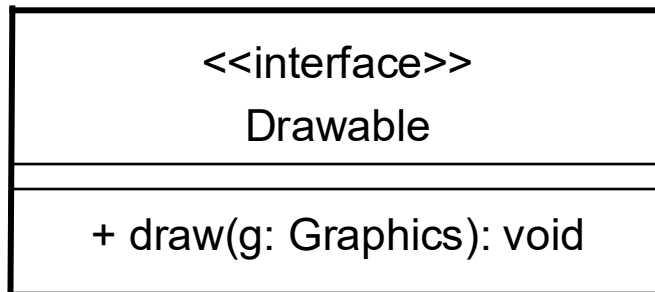
The top compartment shows the name of the object and its class.

The bottom compartment contains a list of the fields and their values.



UML Notation for Interfaces

```
interface Drawable {  
    void draw(Graphics g);  
}
```

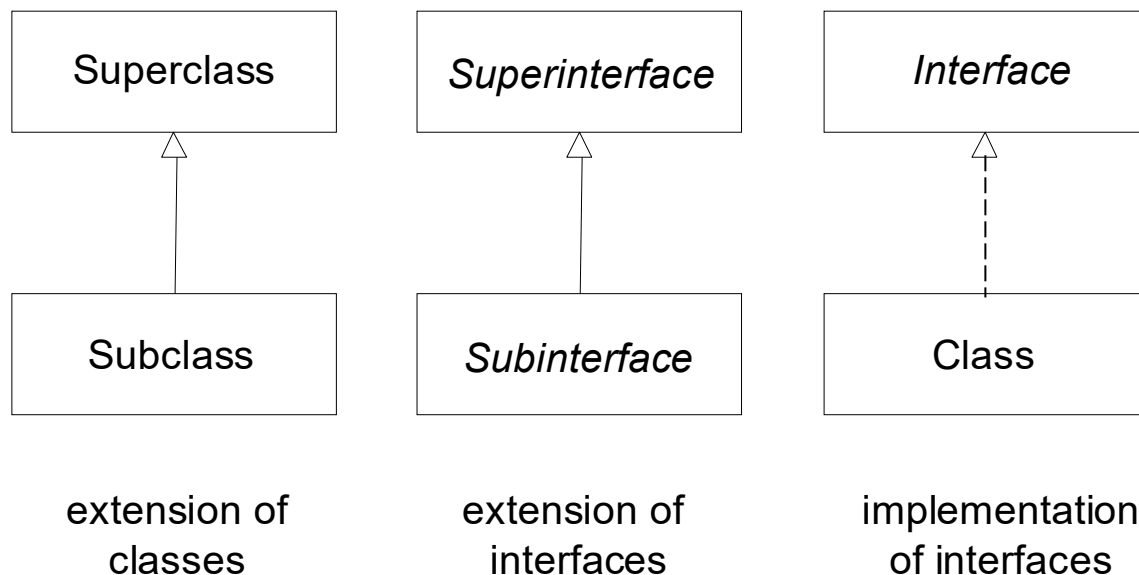


Inheritance in Java

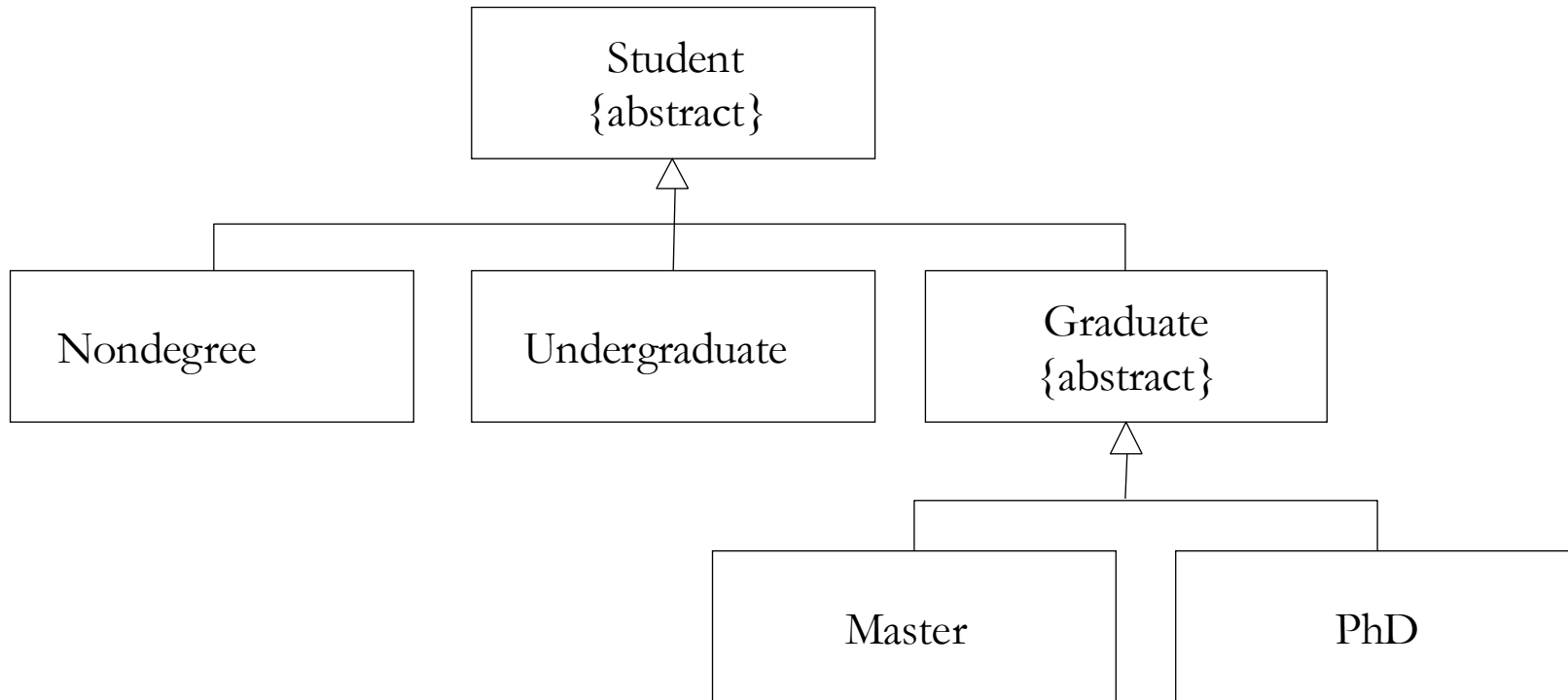
- Important relationship in OO modeling
- Defines a relationship among classes and interfaces.
- Three kinds of inheritances
 - *extension* relation between two classes (*subclasses* and *superclasses*)
 - *extension* relation between two interfaces (*subinterfaces* and *superinterfaces*)
 - *implementation* relation between a class and an interface

Inheritance in UML

- An extension relation is called *specialization* and *generalization*.
- An implementation relation is called *realization*.



Example



Practice Activity

- Draw a UML class diagram showing possible inheritance relationships among classes Person, Employee, and Manager.

Association

- General binary relationships between classes
- Commonly represented as direct or indirect references between classes
- May have an optional label consisting of a name and a direction drawn as a solid arrowhead with no tail.
- The direction arrow indicates the direction of association with respect to the name.



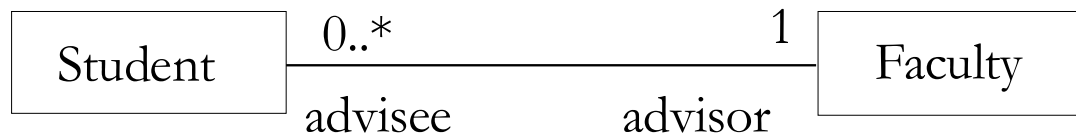
Association (Cont.)

- An arrow may be attached to the end of path to indicate that *navigation* is supported in that direction
- What if omitted?

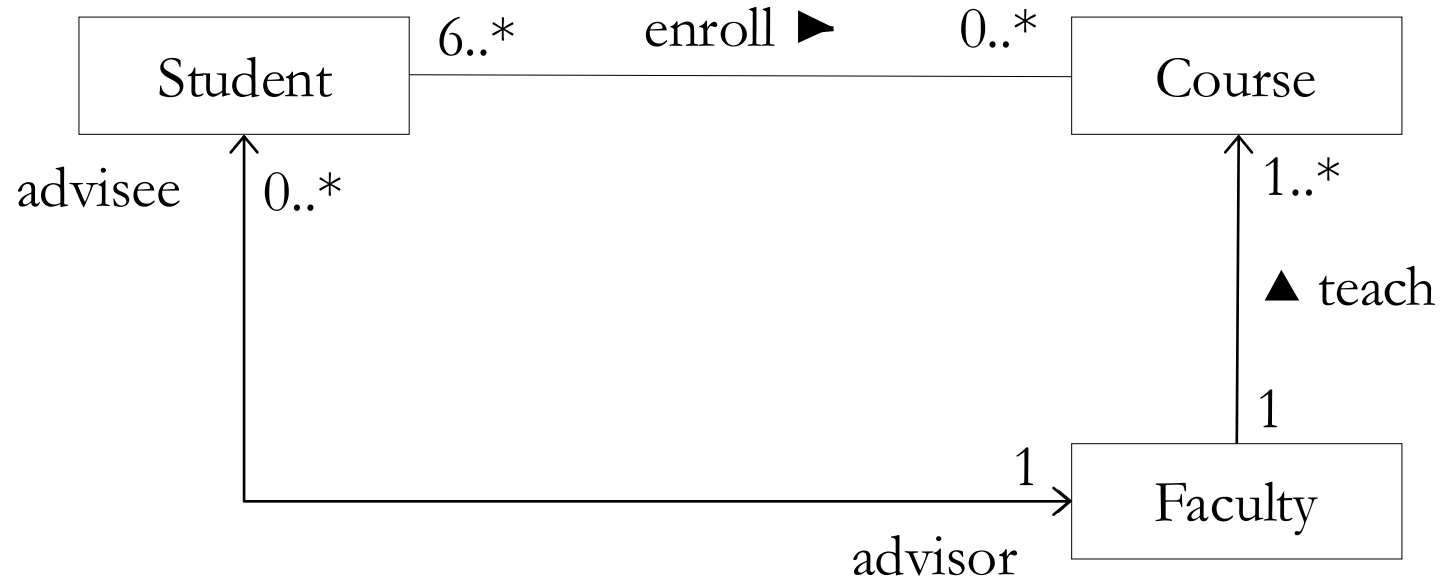


Association (Cont.)

- May have an optional *role name* and an optional *multiplicity specification*.
- The multiplicity specifies an integer interval, e.g.,
 - $m..n$ closed (inclusive) range of integers
 - i singleton range
 - $0..*$ entire nonnegative integer, i.e., 0, 1, 2, ...



Course enrollment example



Practice Activity

- Identify possible relationships among the following classes and draw a class diagram
 - Employee
 - Manager
 - Department

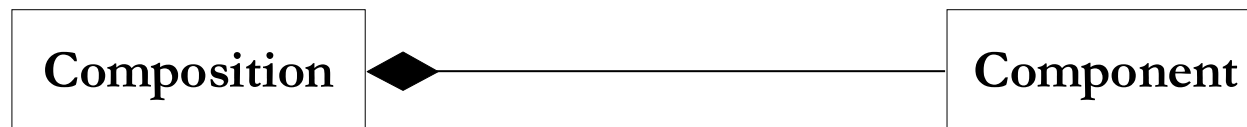
Aggregation

- Special form of association representing *has-a* or *part-whole* relationship.
- Distinguishes the whole (aggregate class) from its parts (component class).
- No relationship in the lifetime of the aggregate and the components (can exist separately).

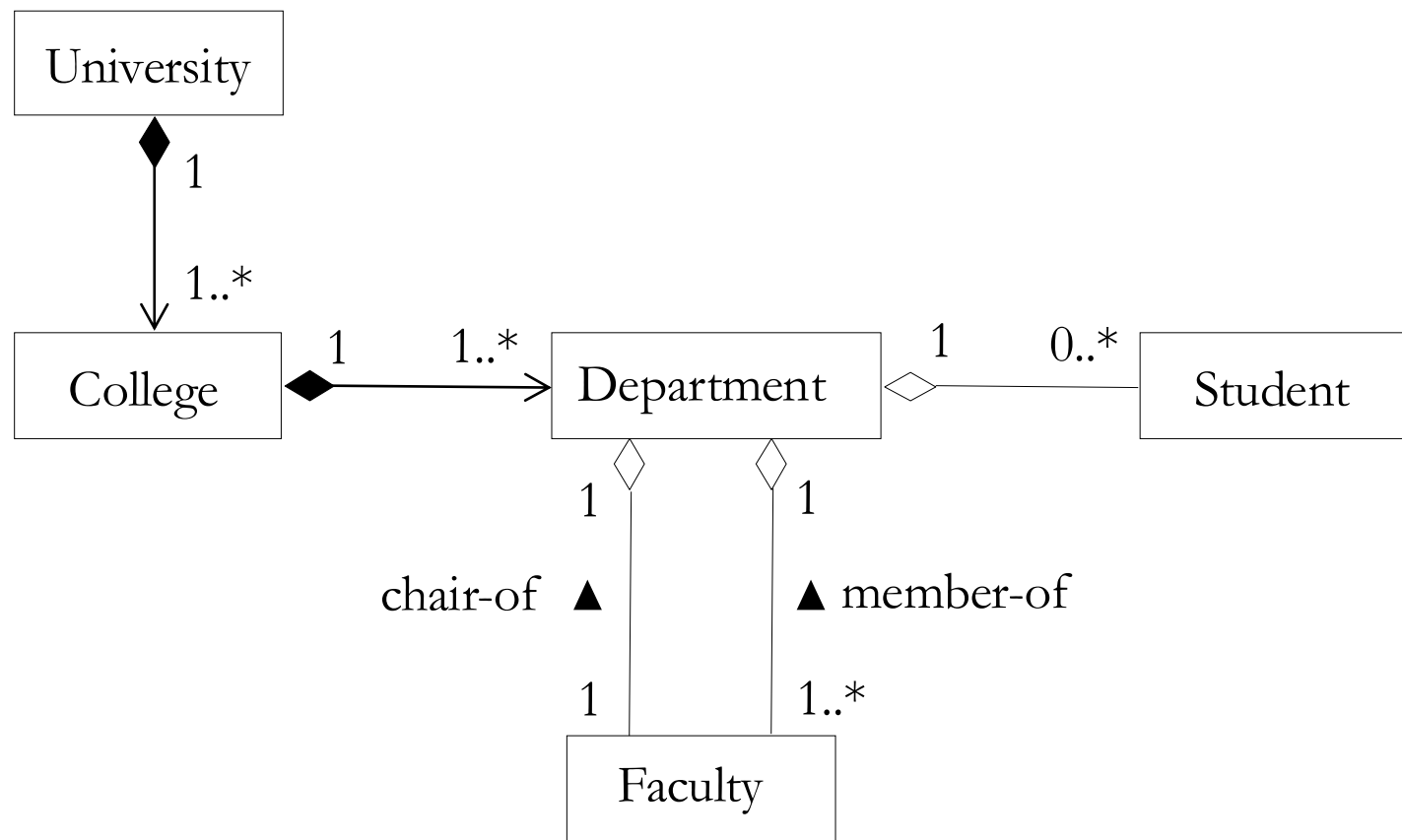


Composition

- Stronger form of aggregation
- Implies exclusive ownership of the component class by the aggregate class
- The lifetime of the components is entirely included in the lifetime of the aggregate (a component can not exist without its aggregate).

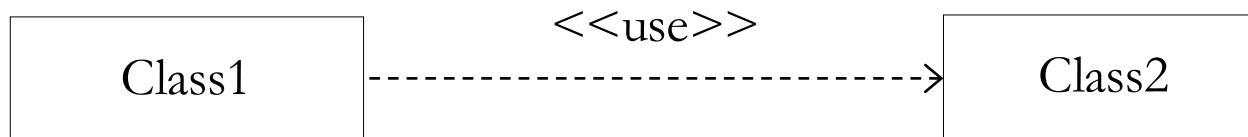


Aggregation Example

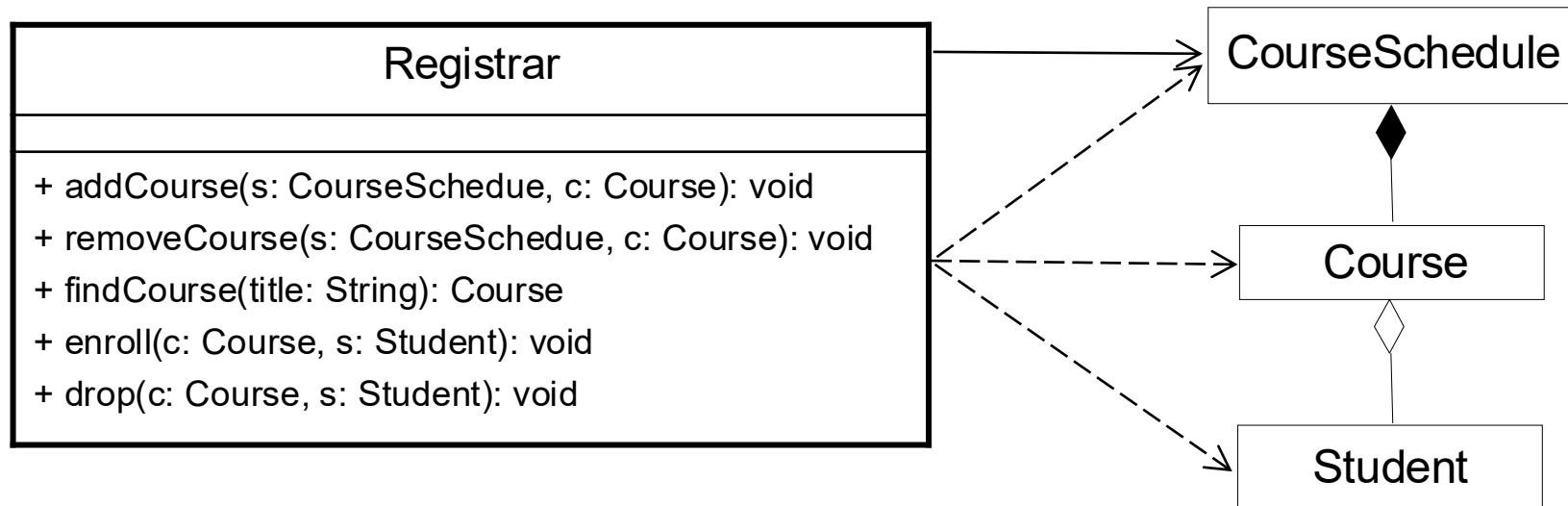


Dependency

- Relationship between the entities such that the proper operation of one entity depends on the presence of the other entity, and changes in one entity would affect the other entity.
- The common form of dependency is the *use* relation among classes.



Dependency Example



Dependencies are most often omitted from the diagram unless they convey some significant information.

Modeling Exercise: E-book Store

Develop an OO model for an e-bookstore. The core requirements of the e-bookstore are to allow its customers to browse and order books, music CDs, and computer software through the Internet. The main functionalities of the system are to provide information about the titles it carries to help customers make purchasing decisions; handle customer registration, order processing, and shipping; and support management of the system, such as adding, deleting, and updating titles and customer information.

1. Identify classes. Classes can represent physical objects, people, organizations places, events, or concepts. Class names should be noun phrases.
2. Identify relevant fields and methods of the classes. Actions are modeled as the methods of classes. Method names should be verb phrases.
3. Identify any inheritance relationships among the classes and draw the class diagram representing inheritance relationships.
4. Identify any association relationships among the classes and draw the class diagram representing association relationships.
5. Identify any aggregation and composition relationships among the classes and draw the class diagram representing dependency relationships.

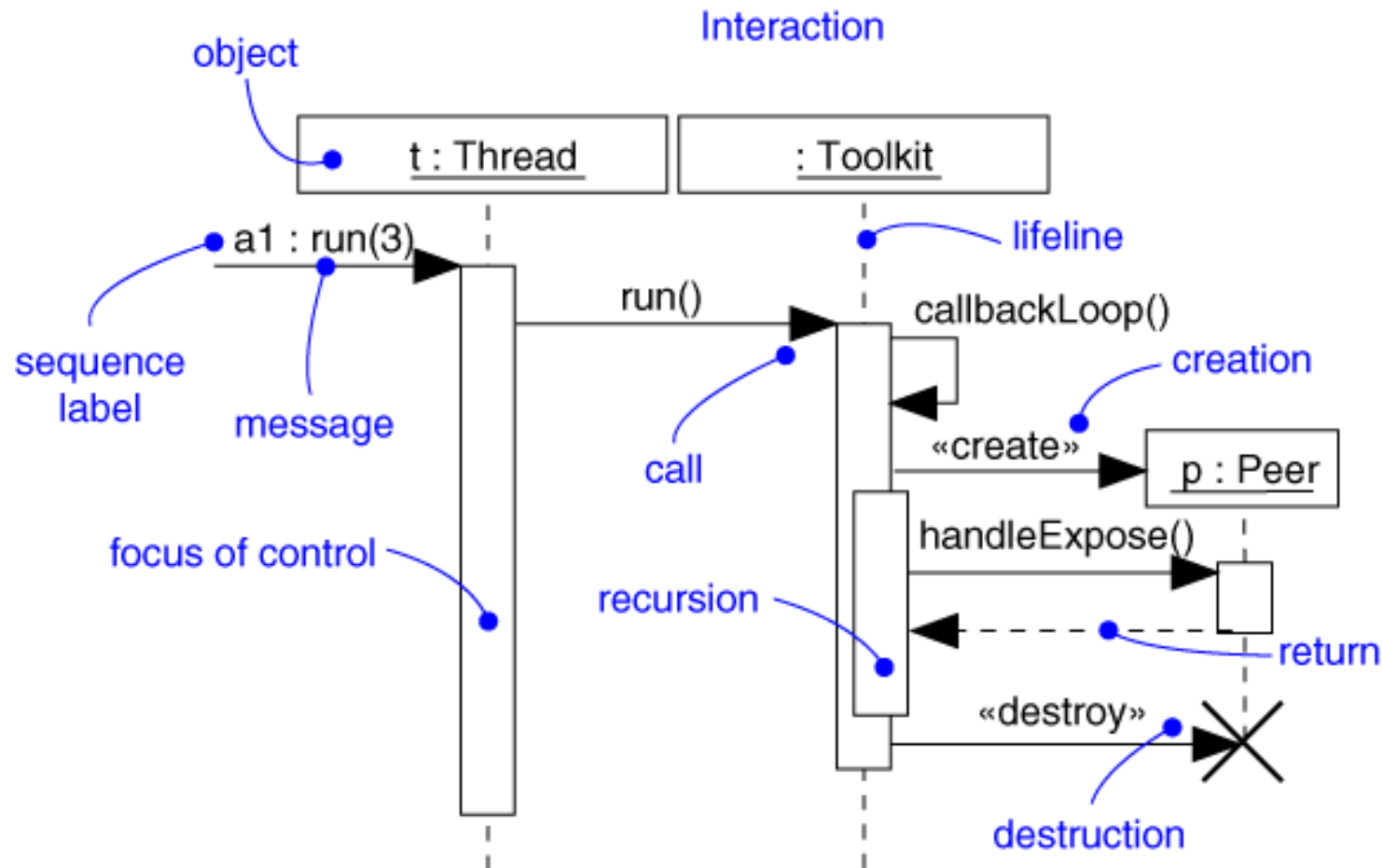
Interaction models

UML Interaction models

- An interaction model shows the interactions that take place between objects in a system
- An interaction “is a behavior that comprises a set of messages exchanged among a set of objects within a context to accomplish a purpose” (UML user guide)
- Interaction models provide a view of system behavior

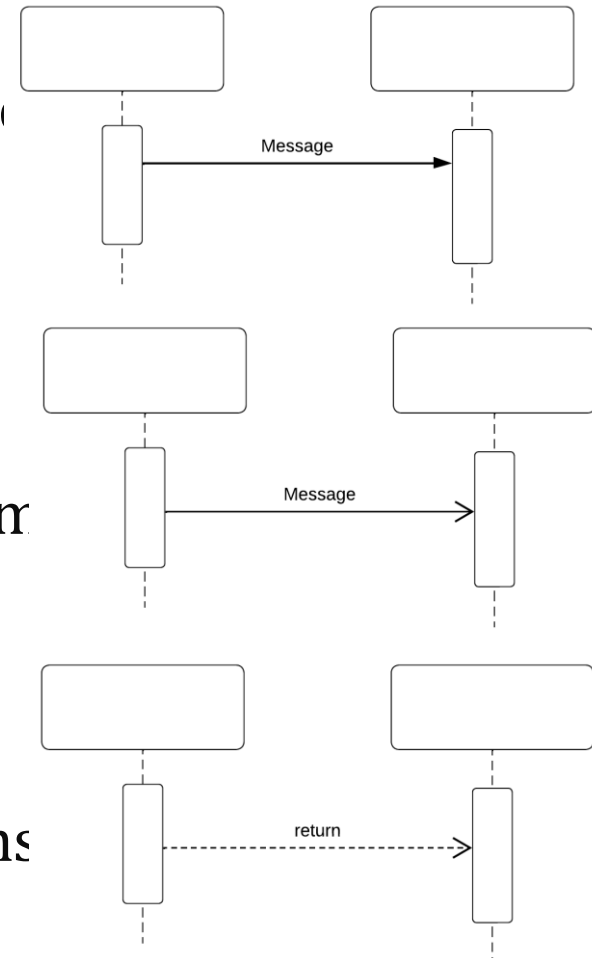
Sequence Diagram

- Captures dynamic behavior (time-oriented)
- Purpose
 - Model flow of control
 - Illustrate typical scenarios



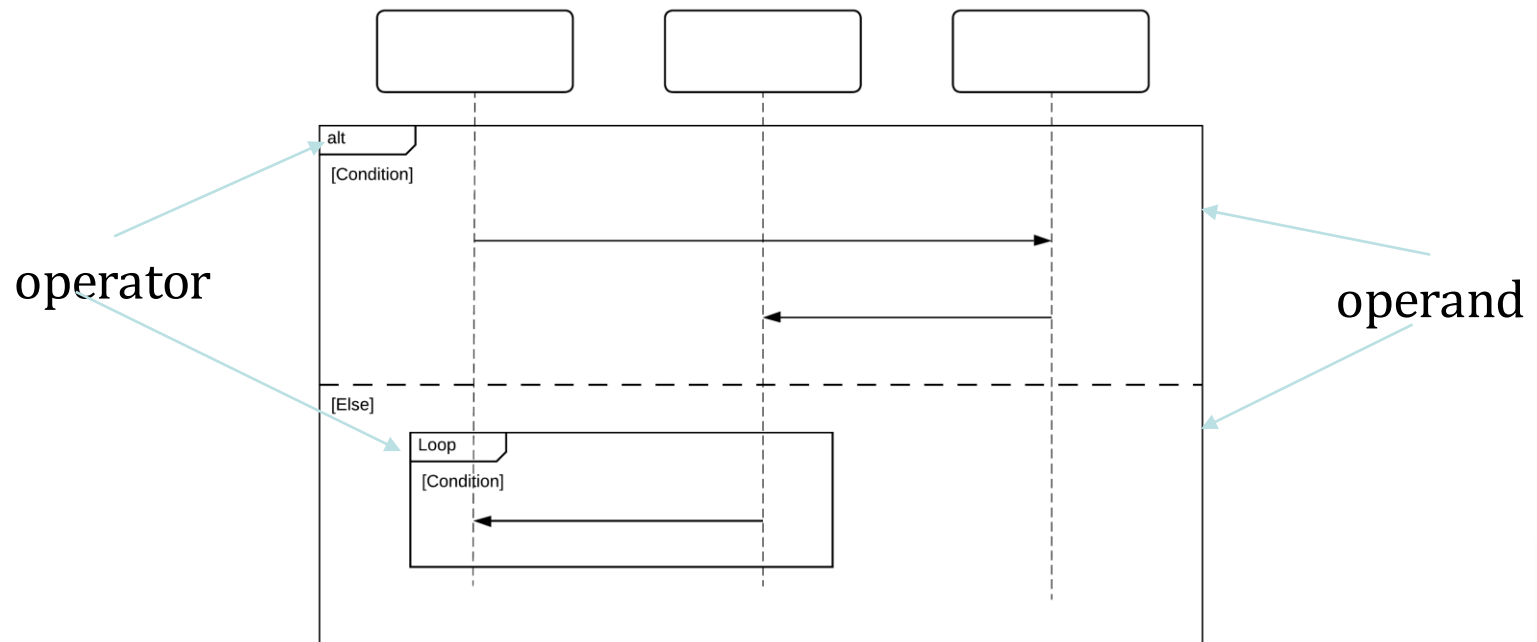
Sequence Diagram – Main Message types

- Synchronous Message
 - Sender waits till the return message is received before next
- Asynchronous Messages
 - Sender does not wait for response message
- Response message
 - Not mandatory in obvious situations



Sequence Diagram – Combined Fragments

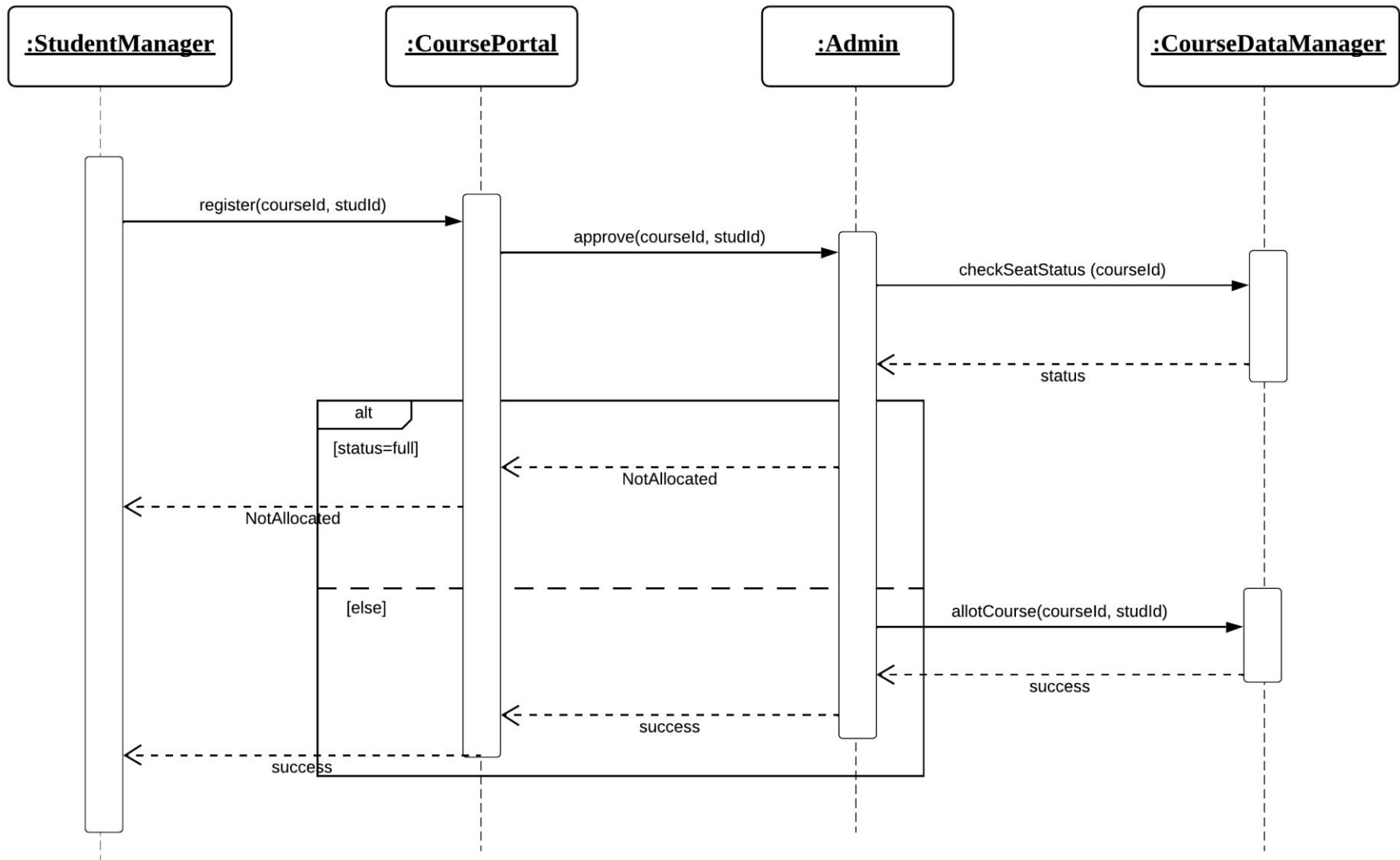
- Model control structures explicitly
- UML sequence diagram supports 12 operators. Three groups
 - Branches and loops, Concurrency and order, Filters and Assertions



Different Operators

Name and Operator	Use
Alternative (alt)	Express alternative execution (if-else)
Optional (opt)	Fragment executes based on guard condition (if)
Break (break)	Execution of a fragment when break condition is met
Loop (loop)	Repeated execution of a fragment
Sequential (seq)	Weak ordering (default model)
Strict (strict)	Interaction with strict order
Parallel (par)	Concurrent execution of sub-scenarios
Critical (critical)	Atomic interactions (no other interactions can affect)
Ignore (ignore)	Irrelevant messages (insignificant messages at runtime)
Consider (consider)	Important messages of the interactions
Negate (neg)	Model invalid interactions, undesirable situations
Assert (assert)	To assert certain interactions (mandatory)

Sequence Diagram – Example



Interaction Modeling Tips

- Set the context for the interaction.
- Express the flow from left to right and from top to bottom.
- Put active objects to the left/top and passive ones to the right/bottom.
- Use sequence diagrams
 - to show the explicit ordering between the stimuli
 - when modeling real-time

Integrating UML into SDLC

- During **requirements**: Use Case Diagrams
- During **design**: Class, Sequence, State Diagrams
- During **implementation**: Map UML to code
- During **testing**: Trace test cases from sequence diagrams

Thank you !